FUNCTIONS & PROCEDURES CS-P2

SUBPROGRAMS: A subprogram is a sequence of instructions in a program that performs a specific task.

TYPES:

- PROCEDURES
- FUNCTION

Why Use Subprograms?

1) Modularity: splits large programs into smaller manageable task
2) Reusibility: Use the same logic in multiplaces, avoiding code repitions.
3) TESTING: Easier to test debug small units.
4) Readibility: Code is easier to understand and maintain

- PROCEDURE vs FUNCTIONS

```
PROCEDURE <name> (parameters)
    //statements
ENDPROCEDURE

<name>( )
```

↳ it is can be any name

↓ optional depends upon requirement

↳ optional (if required)

```
FUNCTION <name> (parameters) RETURNS INTEGER
    // statements
    RETURN <value>
END FUNCTION
```

// Difference main in PROC vs FUNC

Q Write a FUNCTION and a PROCEDURE that take two integers FUNCTION Returns the sum PROCEDURE prints it

FUNCTION

values set: 3, 5

```
FUNCTION AddNum(n1, n2) RETURNS INT
    // local variable
    // statements Part ↓
    DECLARE Sum: INTEGER
    Sum ← n1 + n2
    RETURN Sum
END FUNCTION
```

```
DECLARE sum: INTEGER
sum ← AddNum(3, 5)
OUTPUT sum
```

8

PROCEDURE

not RETURN

Type

```
PROCEDURE AddNum(N1, N2)
    // Statement here ↓
    // local variable
    DECLARE Sum: INTEGER
    Sum ← n1 + n2
    OUTPUT Sum
END PROCEDURE
```

3 5

```
AddNum(3, 5)
DECLARE
```

KEY DIFFERENCES

PROCEDURE

① No Return value

② called will a call eg PRINT( )

③ Used for actions / process

FUNCTIONS

① Returns a value

② called with a expression
sum ← addNum(2,3)

③ Used few calculation

3 Parameters and Arguements:

- **Parameter**: A value used to pass info to a subprogram
- Types: Procedures and Functions can have 0 or more parameters
- Syntax: PROCEDURE Add (a, b)
- Used when: Providing input to the sub program to make it flexible.

- LOCAL & GLOBAL Variables

① **Local Variables**: Declared and exsist only inside the subprogram cannot be accessed outside it

② Global Variable: Exsist outside the subprogram and can be accesed any where.